

SISTEMAS DIGITALES

Simple Linux Programming

"In theory there is no difference between theory and practice. In practice there is."
Yogi Berra

1. Editor

kate is a nice editor for many languages, including the C/C++ programming languages. Besides, it is available for Linux, Windows and macOS. Since we will use it during the semester, it is already installed and configured in the VM. You can, however, install it in your physical machine. For that purpose, it is very convenient that part of the configuration be the same.

Therefore, in Settings → Configure – Kate → Editor Component → Appearance set:

- Dynamic Word Wrap
- Show line numbers

In Settings → Configure – Kate → Editor Component → Editing → General set:

- Insert spaces instead of tabulators
- Tab width: 2 characters
- Remove trailing spaces
- Auto brackets

In Linux you may execute kate (or any program for that matter) by pressing <Alt>+<F2>. A small pop up window will allow you to enter the command; then, press <Enter> to execute it.

2. Compressing/Uncompressing archives

Conventionally, laboratory files would be provided in compressed tar archive files ending in ".tar.gz". To uncompress files of this kind you use the tar and gzip commands. tar can pack many files together into a single disk archive; then, you can use gzip to compress it. The result is a .tar.gz archive file (e.g., file.tar.gz).

To unpack a .tar.gz archive file (e.g., file.tar.gz), use:

```
tar zxvf file.tar.gz
```

Conversely, you can create a .tar.gz archive file (say, file.tar.gz) by packing an entire directory (e.g., dirname, with all its contents) by entering

```
tar zcvf file.tar.gz dirname
```

3. Compilation using the command-line

Change to directory Step0. To execute the `hello1.py` file, you need to run the Python interpreter. To invoke it use the following command line:

```
python
```

After the Python interpreter version, a prompt appears:

```
>>>
```

After the prompt you can type Python statements:

```
>>> print ( "Hello, World!" )
```

or execute a Python program:

```
>>> execfile ("hello1.py")
```

To terminate execution of the Python interpreter, just type:

```
>>> quit ()
```

Instead of running Python programs from inside the interpreter, you can also pass a Python program filename as an argument to the interpreter:

```
python hello1.py
```

For a large program you can compile to byte code the Python program by using:

```
python -m compileall hello1.py
```

This will produce a `hello1.pyc` (a byte-compiled version of `hello1.py`), which runs faster since it can be interpreted more quickly than its ASCII counterpart (`hello1.py`).

Another possibility would be to execute a Python program by means of a runnable script:

```
./hello2.py
```

On the other hand, to execute a C program, you need to compile it first. If your program fits in just

one file, say `hello.c`, you can use the following command line to get the corresponding executable:

```
gcc -Wall hello.c
```

Notice that once the compilation has finished, the prompt appears and apparently, nothing happened. Assuming that there were no errors during the compilation process, if you list the contents of the current directory, there is a new file whose name is `a.out`. That is default filename of the executable binary image. In order to execute it (unless the current directory is in your path) the command line would be:

```
./a.out
```

Now, assuming that you want to use another name for the executable (say `hello`), you should use:

```
gcc -Wall -o hello hello.c
```

In that case you would use the following command line to execute it:

```
./hello
```

`gcc` (GNU C Compiler) has a lot of options. Should you want to check them, please use the following command line:

```
man gcc
```

or

```
info gcc
```

Change to directory `Step1`. There you will find a C source file (`.c`) and a header file (`.h`).

Let's see in detail the compilation process. The first step in the compilation process is to pre-process the source file; that is, strip out any line that does generate code and expand any include directive and macro definitions. You can use the `cpp` script file to pre-process any `.c` file like this:

```
./cpp main.c
```

Basically, the above command is the same as:

```
gcc -E main.c -o main.i
```

The resulting file, `main.i`, has the result of running the pre-processor on `main.c`.

Once the pre-processing has finished, the next step is to translate the pre-processed C source file to assembly code. To do this, use the `gen_asm` script file:

```
./gen_asm main.c
```

Basically, the above command is the same as:

```
gcc -S main.c
```

The result of executing the previous command is `main.s`, which contains the equivalent assembly instructions for `main.c`.

The final step in the compilation is to convert the assembly code into machine language.

The last script file, called `compile`, invokes the entire compilation process; starting with pre-processing; converting to assembly language and, finally to an executable file, which contains machine language.

Change to directory `Step2`. When programming in C, your code usually is written in several files. Let's see how to proceed in those cases. In this particular case, we have two source files. Thus, it would be necessary to use a different strategy: first, you would **compile (generate object files)** from one source file at a time, by using a command line similar to:

```
gcc -Wall -c add.c
```

or

```
gcc -Wall -c main.c
```

Using the `-c` option will create an object (`.o`) file, which is a file with the equivalent machine language of the source code (`.c` file).

To **link** the resulting files one step further is necessary; use the following command line:

```
gcc -Wall -o add2 main.o add.o
```

The linking process will create an executable file (`add2` in this case).

4. Makefile usage

Up to now, the process of compiling, linking, and so on, has been made by hand, which is quite

tedious and prone to error. To cope with that, you can write scripts (or batch files) that automate the process of creating executables (like `compile`); this is a brute force method, however: all the files are recompiled whether or not they need to be.

Fortunately, there is a utility called `make`, that will handle those details. It is designed to aid the programmer in compiling and linking programs: compilation and linking are dependent upon whether a file has been updated since the last compilation. The utility allows you to specify the dependencies of the program file and the source files, and the commands that generate the program from its source. The file that contains the rules used by `make` to decide how to build the program is called `Makefile`. A `makefile` consists of a set of dependencies and rules. A dependency has a target (a file to be created) and a set of source files upon which it is dependent. The rules describe how to create the target from the dependent files. Commonly, the target is a single executable file.

The `makefile` is read by the `make` command, which determines the target file or files that are to be created and then compares the dates and times of the source files to decide which rules need to be invoked to construct the target. Often, other intermediate targets have to be created before the final target can be made. The `make` command uses the `makefile` to determine the order in which the targets have to be made and the correct sequence of rules to invoke.

The dependencies specify how each file in the final application relates to the source files; rules describe how to create a target. Go to the `Step3` directory and check the `makefiles` there.

`make -f mkfile?`

where `mkfile?` can be one of

`mkfile1` - A simple `makefile`

`mkfile2` - A `makefile` with macros

`mkfile3` - A `makefile` with macros and multiple targets

In the `Step4` directory you can find a generic `Makefile`.

5. Libraries

As you progress in your coding, many object modules are created; some of them could be useful not just for the current project, but also for other forecoming projects. In that case, it would be nice to have a library with those object modules, so that the next time, the linking process would be faster and easier.

In Linux, there are two types of libraries: static and dynamic (shared, equivalent to Windows DLLs). The first ones are meant for those cases in which the executable image contains all the required

functionality inside. On the other hand, dynamic libraries, reside outside of the binary image a separate files, which allows several executable images share the functionality provided by a single copy of a given dynamic library.

Naming of static libraries adhere to the following convention:

`lib*.a`

while dynamic ones use:

`lib*.so`

or

`lib*.sa`

In those cases, maybe your header files remain in a particular directory, say `include`. Since, by default, the compiler searches for user include files within the current directory, it is necessary to point out where the include files are:

```
gcc -Wall -I./include program1.c
```

Now, suppose your program is composed of three source files: `main.c`, `add.c` and `sub.c`. You could use the Makefile in directory Step5 to build an executable.

When your programs frequently use common functions/procedures, modifying the makefile or even worst, using the commands needed to build your application is rather cumbersome. In those cases, it is very useful to create a library:

```
ar crv libops.a add.o sub.o
```

The previous command would create an archive library called `libops.a`, replacing any modules if necessary, and it would be verbose when doing its duty. To create a contents table for the library, use the following command line:

```
ranlib libops.a
```

Once you have done this, a library called `libops.a` is ready to use. Using it is easy, you just add it to the object files list:

```
gcc -Wall -o addsub2 main.o libops.a
```

In the previous case, it is assumed that `libops.a` is in the current directory. You could put it in

whatever directory you like however. If `libops.a` resides in the `lib` directory, use:

```
gcc -Wall -o addsub3 main.o -Llib -lops
```

where `-L` points out to the directory to be searched and `-l` indicates the name of the library to be used. Notice that the `lib` prefix is removed. Sometimes it is useful to list symbols from object files (a library is a collection of object modules):

```
nm --print-file-name addsub3 | less
```

or

```
nm --print-file-name libops.a
```

To print shared library dependencies, use:

```
ldd addsub3
```